

OpsMate Docker Deployment Project

Containerizing and Deploying OpsMate Service Using Docker and AWS

Project Report

Prepared by	Saad Momin
Project	OpsMate
Platform	Docker and Amazon Web Services
AWS Region	ap-south-1
Core Services	EC2, ECR, ECS, Fargate, CloudWatch

September 2026

Contents

1. Project Overview
2. Objectives
3. Technology Stack
4. Docker Fundamentals and Architecture
5. Docker Installation and Daemon Management
6. OpsMate Application and Dockerfile
7. Docker Image and Container Management
8. Docker Compose and MySQL Deployment
9. Amazon ECR Integration
10. Amazon ECS and AWS Fargate Deployment
11. Application Update and Redeployment
12. Problems Faced and Troubleshooting
13. Security Considerations
14. Optimization and Future Improvements
15. Project Evidence and Deliverables
16. Conclusion

1. Project Overview

OpsMate is a DevOps mini-project that demonstrates an end-to-end container deployment workflow. The application was containerized using Docker on an Ubuntu EC2 instance, deployed locally as a standalone container, extended to a multi-container deployment with MySQL using Docker Compose, stored in Amazon Elastic Container Registry (ECR), and deployed on Amazon Elastic Container Service (ECS) using AWS Fargate.

The project also demonstrates application versioning and redeployment by building a Version 2 image, pushing it to ECR, creating a new ECS task definition revision, and updating the running ECS service.

2. Objectives

- Understand virtualization, containerization, Docker architecture, Docker Engine, Docker daemon, images, containers, and registries.
- Install and manage Docker Engine on an Ubuntu EC2 instance.
- Create a Dockerfile for the OpsMate Node.js application and build a reusable Docker image.
- Run and manage Docker containers, inspect configuration, review logs, and clean unused resources.
- Deploy the application and a MySQL database using Docker Compose, networking, environment variables, and persistent storage.
- Push Docker images to a private Amazon ECR repository.
- Deploy the application using Amazon ECS and AWS Fargate.
- Update the application and redeploy a new container image version.

3. Technology Stack

Category	Technology
Cloud	Amazon Web Services (AWS)
Compute	Amazon EC2, AWS Fargate
Containers	Docker, Docker Compose
Registry	Amazon ECR
Orchestration	Amazon ECS
Application	Node.js, Express.js
Database	MySQL 8.0
Logging	Docker logs, Amazon CloudWatch
Operating System	Ubuntu Linux
Version Control	Git and GitHub

4. Docker Fundamentals and Architecture

Docker uses a client-server architecture. The Docker client sends commands to the Docker daemon, which builds images, creates and manages containers, and communicates with registries. A Docker image is an immutable package containing the application and its dependencies, while a container is a running instance of that image.

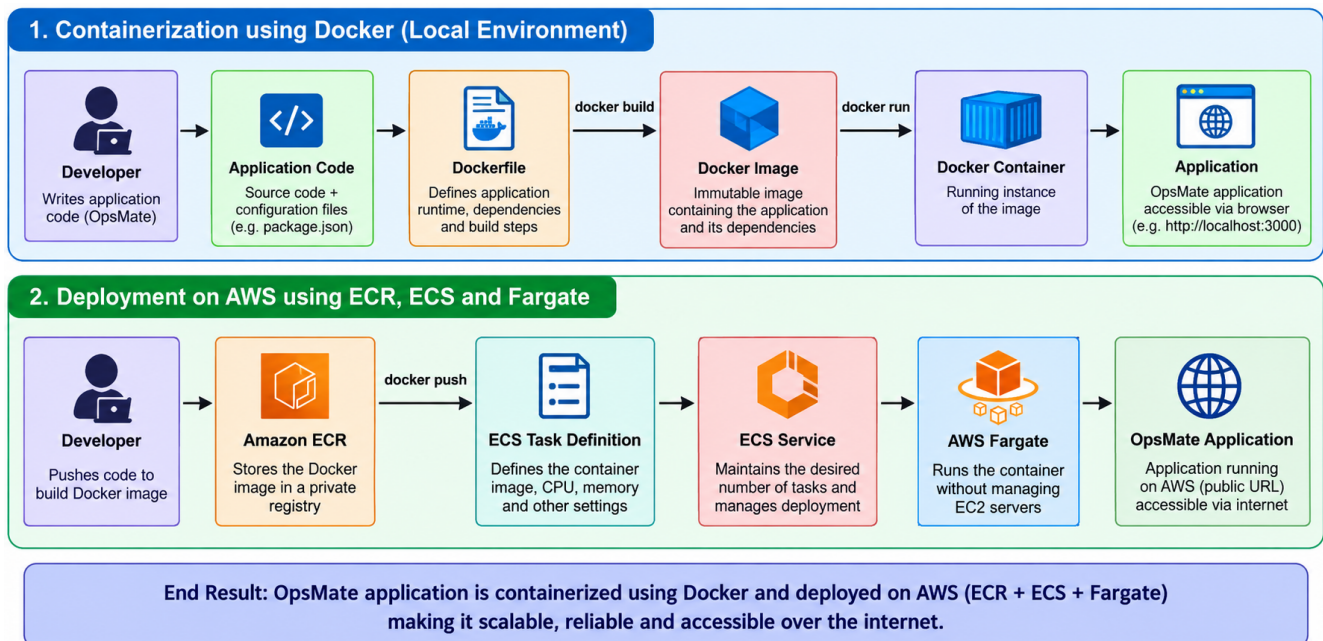


Figure 1. OpsMate Docker and AWS deployment architecture.

5. Docker Installation and Daemon Management

An Ubuntu EC2 instance was used as the Docker server. Docker Engine was installed using the Ubuntu package manager and verified from the command line.

```
sudo apt update
sudo apt install docker.io -y
docker --version
sudo systemctl status docker
```

Docker service management was practiced using start, stop, restart, and enable operations. The Ubuntu user was added to the Docker group so Docker commands could be executed without sudo. Docker functionality was finally verified using the hello-world container.

```
sudo systemctl stop docker
sudo systemctl start docker
sudo systemctl restart docker
sudo systemctl enable docker
sudo usermod -aG docker $USER
docker run hello-world
```

6. OpsMate Application and Dockerfile

OpsMate is a lightweight Node.js application built with Express.js and exposed on port 3000. The project uses package.json to define the application metadata, start command, and dependencies. MySQL connectivity is provided through the mysql2 package for the Docker Compose deployment.

Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY app/package*.json ./
RUN npm install
COPY app/ ./
EXPOSE 3000
CMD ["npm", "start"]
```

The node:20-alpine base image provides the Node.js runtime inside the container, so Node.js does not need to be installed directly on the EC2 host for the application to run.

7. Docker Image and Container Management

The initial OpsMate image was built and tagged as opsmate:v1. The image was listed and inspected to verify its configuration.

```
docker build -t opsmate:v1 .
docker images
docker inspect opsmate:v1
```

The application container was started in detached mode with host port 3000 mapped to container port 3000. The application was then accessed from a browser using the EC2 public IP.

```
docker run -d -p 3000:3000 --name opsmate-container opsmate:v1
docker ps
docker logs opsmate-container
docker inspect opsmate-container
```

Container lifecycle operations including stop, start, restart, remove, and recreation were demonstrated. Docker disk usage and build-cache cleanup were also checked.

```
docker stop opsmate-container
docker start opsmate-container
docker restart opsmate-container
docker rm opsmate-container
docker system df
docker builder prune
```

8. Docker Compose and MySQL Deployment

Docker Compose was used to deploy two services: the OpsMate application and a MySQL 8.0 database. The Compose configuration defined service dependencies, environment variables, a custom network, persistent database storage, and port mapping.

The final GitHub-ready Compose file references variables from a local .env file rather than publishing database passwords directly in source control. The .env file is excluded using .gitignore, while config/environment.example documents the required variable names with placeholder values.

```
docker compose up -d
docker compose ps
docker compose logs
docker compose stop
docker compose start
docker compose down
```

Database connectivity was verified through the /db application endpoint. A named Docker volume was used to preserve MySQL data when the Compose stack was stopped or recreated.

9. Amazon ECR Integration

A private Amazon ECR repository named `opsmate` was created in the `ap-south-1` region. Docker authenticated to ECR using the AWS CLI, after which the local image was tagged with the ECR repository URI and pushed.

```
aws ecr get-login-password --region ap-south-1 | docker login --username AWS
--password-stdin <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com

docker tag opsmate:v1 <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/opsmate:v1
docker push <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/opsmate:v1
```

Version 1 and Version 2 of the application were stored in ECR, providing versioned container artifacts for ECS deployment.

10. Amazon ECS and AWS Fargate Deployment

The existing ECS cluster `opsmate_cluster` was used for the deployment. An ECS task definition named `opsmate-task` was configured for AWS Fargate with Linux/X86_64, 0.25 vCPU, 0.5 GB memory, container port 3000, the ECR image URI, and CloudWatch logging.

An ECS service named `opsmate-service` was created with a desired task count of one. Networking used a VPC, public subnet, security group, and public IP. For the project demonstration, inbound TCP port 3000 was allowed so the application could be verified directly in a browser.

- ECS cluster: `opsmate_cluster`
- Task definition family: `opsmate-task`
- Service: `opsmate-service`
- Launch type: AWS Fargate
- Desired tasks: 1
- Container port: 3000

The Fargate task reached the `RUNNING` state and the OpsMate application was successfully accessed through the task's public endpoint.

11. Application Update and Redeployment

The application was updated to display OpsMate Version 2 and a message confirming successful deployment through Amazon ECS Fargate. A new Docker image was built as `opsmate:v2`, tagged, and pushed to ECR.

```
docker build -t opsmate:v2 .
docker tag opsmate:v2 <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/opsmate:v2
docker push <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/opsmate:v2
```

A new revision of the `opsmate-task` task definition was created with the `v2` ECR image. The ECS service was updated to use the new revision. After the replacement Fargate task became healthy and running, Version 2 was verified in the browser.

12. Problems Faced and Troubleshooting

Problem	Cause / Observation	Resolution
Port 3000 already allocated	The standalone OpsMate container was still running on port 3000 when the compose started.	Stopped port 3000 when the compose started.
Compose plugin package unavailable	Docker-compose-plugin could not be located	Installed the available Docker Compose v2 package and used that.
Application container exited	A syntax error was present in the first line of <code>check.js</code>	Checked docker compose logs, corrected the JavaScript syntax error.
AWS CLI missing	The aws command was not installed on the EC2 instance.	Installed AWS CLI, configured credentials/region as needed, and ran <code>aws --version</code> .
Incorrect docker push syntax	docker push was initially given the local and registry image as argument.	Revised the local image just then pushed the single complete ECR image.

13. Security Considerations

- AWS access keys and secret keys must never be stored in application source code or committed to GitHub.
- IAM roles and least-privilege permissions should be preferred where possible.
- Database passwords and other secrets should not be hardcoded in public source files; production deployments should use a managed secret store.
- The local `.env` file is excluded from Git using `.gitignore`, and a placeholder `environment.example` file is provided for documentation.
- Opening port 3000 to 0.0.0.0/0 is acceptable only for a short project demonstration; production access should be restricted and normally fronted by a load balancer and HTTPS.
- Base images and application dependencies should be regularly updated and scanned for vulnerabilities.

14. Optimization and Future Improvements

- Continue using a small base image such as `node:20-alpine` to reduce image size.
- Use `.dockerignore` to keep unnecessary files out of the build context.
- Use Amazon RDS rather than a containerized MySQL database for a production AWS deployment.
- Place the ECS service behind an Application Load Balancer and enable HTTPS.
- Store secrets in AWS Secrets Manager or Systems Manager Parameter Store.
- Add ECS Service Auto Scaling, CloudWatch alarms, and improved observability.
- Automate build, test, ECR push, and ECS deployment using a CI/CD pipeline.
- Add container health checks and stronger application readiness checks.

15. Project Evidence and Deliverables

The GitHub project repository contains the application source, `Dockerfile`, `compose.yaml`, configuration example, Docker management scripts, `README`, deployment report, screenshots, and architecture documentation.

- 01-docker-installation.png
- 02-docker-daemon.png
- 03-docker-hello-world.png

- 04-task2-final-verification.png
- 05-dockerfile.png
- 06-docker-image.png
- 07-docker-container.png
- 08-application-running.png
- 09-compose-file.png
- 10-docker-compose.png
- 11-database-connectivity.png
- 12-ecr-repository.png
- 13-ecr-image.png
- 14-ecs-cluster.png
- 15-ecs-task.png
- 16-fargate-running-task.png
- 17-fargate-application.png
- 18-fargate-v2-deployment.png

The repository also includes `Docker-Architecture.png` and `docker-deployment-report.txt` as supporting documentation.

16. Conclusion

The OpsMate project successfully demonstrates a complete Docker and AWS container deployment workflow. Docker Engine was installed and managed on Ubuntu, the Node.js application was packaged into a Docker image, container lifecycle operations were practiced, and Docker Compose was used to run the application with MySQL.

The container image was then stored in Amazon ECR and deployed through Amazon ECS using AWS Fargate. Finally, the application was updated to Version 2 and redeployed through a new task definition revision. The project therefore covers containerization, multi-container orchestration, image registry integration, serverless container deployment, troubleshooting, security considerations, and application version updates.